

Q1) Differentiate Between Informed Search and Uninformed Search

In Artificial Intelligence, **search algorithms** are used to navigate through problem spaces to find solutions. These algorithms are broadly categorized into **uninformed (blind)** and **informed (heuristic)** search strategies.

Aspect	Uninformed Search	Informed Search
Definition	Search strategies that do not use any domain-specific knowledge beyond the problem definition.	Search strategies that use additional domain knowledge (heuristics) to find solutions efficiently.
Alternative Name	Blind Search	Heuristic Search
Heuristic Use	No	Yes
Guidance	Explores the search space blindly	Guided by a heuristic function estimating the cost to reach the goal
Efficiency	Less efficient; may explore many irrelevant paths	More efficient; focuses on more promising paths
Time & Space	May consume more time and memory	Generally consumes less time and memory
Optimality	Depends on the specific algorithm (e.g., UCS is optimal)	Can be optimal if an admissible and consistent heuristic is used (e.g., A*)
Examples	Breadth-First Search (BFS), Depth-First Search (DFS), Uniform Cost Search (UCS)	A* Search, Greedy Best-First Search

Q2) Difference Between BFS and DFS with Advantages & Disadvantages

- **Breadth-First Search (BFS):**
An algorithm that explores all the nodes at the present depth level before moving to the next level. It uses a **queue** (FIFO) data structure.
- **Depth-First Search (DFS):**
An algorithm that explores as far as possible along each branch before backtracking. It uses a **stack** (LIFO) or **recursion**.

Aspect	Breadth-First Search (BFS)	Depth-First Search (DFS)
Traversal Method	Explores all nodes level by level (breadth-wise)	Explores as far as possible along each branch (depth-wise)
Data Structure Used	Queue	Stack (or recursion)
Completeness	Complete (if branching factor is finite)	Not complete (may get stuck in infinite paths)
Optimality	Optimal (for unweighted graphs)	Not optimal
Time Complexity	$O(b^d)$	$O(b^d)$
Space Complexity	$O(b^d)$	$O(b \cdot d)$
Best Used When	You need the shortest path or in shallow graphs	Memory is limited or solutions are deep in the tree

Where b = branching factor, d = depth of the shallowest solution.

Advantages and Disadvantages

Breadth-First Search (BFS)

- ✔ Advantages:
- Guarantees shortest path in unweighted graphs.
 - Complete; it will find a solution if one exists.
- ✘ Disadvantages:
- High memory usage (stores all nodes at current level).
 - Slow for large or deep graphs.

Depth-First Search (DFS)

- ✔ Advantages:
- Requires less memory compared to BFS.
 - Can find solutions without checking every node (in favorable cases).
 - Easy to implement using recursion.
- ✘ Disadvantages:
- May go deep into infinite paths (incomplete).

- Does not guarantee the shortest path.
- Can miss a shallow solution.

Q3) Differentiate Between Strong AI and Weak AI

- **Strong AI (Artificial General Intelligence - AGI):**
Refers to machines that possess the ability to understand, learn, and apply intelligence at a human level. It can perform any intellectual task that a human can do.
- **Weak AI (Narrow AI):**
Refers to AI systems designed and trained for a specific task. These systems simulate intelligence but do not possess consciousness or real understanding.

Comparison Table:

Aspect	Strong AI	Weak AI
Also Known As	Artificial General Intelligence (AGI)	Narrow AI
Intelligence Level	Human-level or beyond	Task-specific intelligence
Scope	Can perform a wide range of tasks	Limited to specific tasks
Learning Ability	Self-aware, capable of reasoning and generalization	Learns from data for specific tasks only
Example	Still theoretical (not yet achieved)	Siri, Alexa, Google Assistant, ChatGPT, self-driving cars
Consciousness	Assumed to have consciousness and understanding	No consciousness or understanding
Autonomy	Fully autonomous decision-making and learning	Limited autonomy, based on predefined algorithms

Q4) Differentiate Between Single-Agent and Multi-Agent Systems

- **Single-Agent System:**
A system in which **only one agent** operates in the environment to achieve its goal, without interaction with other agents.
- **Multi-Agent System:**
A system composed of **multiple interacting agents**, which may cooperate or compete to achieve individual or collective goals.

Comparison Table:

Aspect	Single-Agent System	Multi-Agent System
Number of Agents	Only one agent	Two or more agents
Interaction	No interaction with other agents	Agents may interact, communicate, cooperate, or compete
Complexity	Simpler to design and manage	More complex due to coordination and conflict handling
Decision Making	Agent makes decisions independently	Decisions may be based on shared knowledge or negotiation
Examples	Pathfinding robot, Chess-playing AI	Autonomous vehicles in traffic, Online trading systems
Environment Type	Typically simpler or fully observable	Often dynamic, partially observable, and unpredictable
Goal Orientation	Focused on a single objective	May involve individual and shared objectives

Q5) What Are the Types of Agents in Artificial Intelligence?

In AI, an **agent** is anything that can perceive its environment through sensors and act upon that environment through actuators. There are **five main types of agents**, each more advanced than the last in terms of capabilities and intelligence.

Types of Agents in AI:

1. **Simple Reflex Agent**
 - **Definition:** Acts only based on the current percept, ignoring the history.
 - **Working:** Uses condition-action rules ("if-then").
 - **Limitation:** Cannot handle complex environments or remember past actions.
 - **Example:** Thermostat, light switch system.
2. **Model-Based Reflex Agent**
 - **Definition:** Maintains an internal model of the world to keep track of changes.

- **Working:** Considers current percept and internal state for decision-making.
- **Advantage:** Can handle partially observable environments.
- **Example:** Self-parking car that remembers obstacles.

3. 3. Goal-Based Agent

- **Definition:** Acts to achieve specific goals.
- **Working:** Chooses actions that bring it closer to a predefined goal.
- **Advantage:** More flexible and purposeful than reflex agents.
- **Example:** GPS navigation system finding the best route.

4. 4. Utility-Based Agent

- **Definition:** Chooses actions based on a utility function that ranks preferences.
- **Working:** Tries to maximize the agent's happiness or usefulness.
- **Advantage:** Handles conflicting goals and preferences better.
- **Example:** Intelligent shopping bot comparing price, delivery, and reviews.

5. 5. Learning Agent

- **Definition:** Can learn from experience and improve its performance over time.
- **Components:** Learning element, performance element, critic, and problem generator.
- **Advantage:** Adapts to dynamic environments and becomes more effective.
- **Example:** AI in video games that adapts to player behavior.

Autonomous agent Scenario based question

✓ Scenario 1: Autonomous Vacuum Cleaner Agent

Problem Statement:

Design an intelligent autonomous vacuum cleaner agent capable of efficiently cleaning a house. The agent must navigate around furniture and obstacles, detect dirty areas, avoid repetitive cleaning, and recharge itself when the battery is low — all without human intervention.

Solution:

The autonomous vacuum cleaner operates in a **partially observable**, **dynamic**, and **stochastic** environment. It is a **goal-based reactive agent** with the following characteristics:

- **Percepts:** Dirt sensors, obstacle detectors, map boundaries, battery level.
- **Actions:** Move forward, turn left/right, clean, return to charging dock.
- **Agent Function:**
 - If dirt is detected → Clean
 - If obstacle is detected → Turn/Change direction
 - If battery < 20% → Return to charging dock
 - Else → Continue cleaning path using DFS or A* algorithm
- **Features:**
 - Path optimization using memory or mapping techniques.
 - Dynamic obstacle avoidance.
 - Auto-recharging and resuming task post recharge.

✓ Scenario 2: Autonomous Book Library Agent

Problem Statement:

Build an autonomous library management agent that helps users search for books, recommend titles based on their interests, manage check-in/check-out processes, and keep real-time track of book availability in a digital library system.

Solution:

This agent functions in a **fully observable**, **static**, and **deterministic** environment. It is a **utility-based agent** with the following capabilities:

- **Percepts:** User search input, borrowing history, current availability of books.
- **Actions:** Search, recommend, issue, return, notify.
- **Agent Function:**
 - Search for books using metadata and indexing.

- Recommend books using collaborative filtering based on user preferences.
- Automate issue/return process and update book status.
- Notify users about due dates or similar book suggestions.
- **Features:**
 - Real-time availability updates.
 - Personalized user experience through recommendation system.
 - Integration with user database for borrowing history.

✓ **Scenario 3: Autonomous Online Tutor Agent**

Problem Statement:

Develop an autonomous online tutor agent that can assess a student's learning level, deliver personalized lessons, provide real-time feedback, and adapt the teaching content dynamically to enhance the student's understanding.

Solution:

This agent operates in a **partially observable, dynamic, and stochastic** environment. It is a **learning agent** that incorporates **machine learning** to personalize instruction.

- **Percepts:** Student responses, time taken per question, accuracy, quiz results.
- **Actions:** Deliver lectures, quiz, give feedback, adjust difficulty, suggest revisions.
- **Agent Function:**
 - Analyze student input to identify knowledge gaps.
 - Adaptively select content and teaching methods (e.g., visual, textual).
 - Provide instant feedback and hints.
 - Track performance and adjust learning path accordingly.
- **Features:**
 - Personalized learning paths.
 - AI-based assessment and feedback.
 - Motivation through gamification and rewards.